

Club Penguin Island Cinematography System Technical Manual

Overview

The Club Penguin Island cinematography system is a modular camera framework for controlling viewpoint logic transitions constraints and special behaviors in a three dimensional environment Its design splits camera motion into separate layers including goal positioning aiming and glancing which allows developers to compose complex and cinematic shots in a maintainable way

The system defines these key layers

Goal The desired position in world space where the camera should move toward

LookAt The point the camera should aim its forward direction toward

Glance A small orientation adjustment applied on top of the aim to introduce subtle reactive motion

Each of these layers is handled by a specific group of scripts collaborating through a Setup structure This Setup structure carries the intermediate positions and rotations as the camera logic is processed every frame

Core Components

CameraController

This is the heart of the system It manages the activation and deactivation of camera logic components including goal planners framers glancers and constraints The CameraController maintains a priority ordering for which camera is active and it stores transitions to other cameras through

CameraControllerTransition It also takes care of deactivating other camera components when disabled and reactivating them as needed

On awake the CameraController collects references to its goal planners constraints framers and glancers then disables them until it is activated It also discovers transitions to other CameraControllers through the CameraControllerTransition components on the same GameObject When the controller is enabled or disabled it dispatches events to reset or reconfigure the camera logic

The CameraController exposes the property IsFinished which indicates if its planners and framers are done performing their tasks usually used to allow smooth transitions

CameraControllerTransition

A CameraControllerTransition defines how the CameraController switches to another CameraController including how fast the transition should occur The transition can happen instantly as a cut with a maximum speed or using a curve The script supports specifying maximum movement and

aim speeds and allows blending movement to the next camera smoothly. It draws helpful gizmos in the Unity editor to visualize the transition paths and icons so designers can see where cameras switch.

GoalPlanner

GoalPlanners determine the desired world space position for the camera goal. A goal is the ideal position of the camera separate from its orientation. The system includes several GoalPlanner implementations for different situations.

HeightOffsetGoalPlanner offsets the camera goal vertically from the focus target to achieve higher or lower perspectives.

FixedOffsetGoalPlanner provides an offset in global, local, or focus local space so the camera positions itself with respect to a fixed offset around the focus target.

GoalPlanners are invoked each frame through their Plan method with the Setup structure to write their calculated position into SetupGoal.

Framer

Framers adjust the look at target of the camera so that a desired object or point is kept in frame. One of the key framers is LookAheadFramer which offsets the look at point based on the focus velocity. This means the faster a character moves the further ahead the camera looks, providing better anticipation of motion.

LookAheadFramer has special handling for keyboard on screen cases, adjusting the offset to accommodate for UI when typing. Its Aim method is called with the Setup structure to set SetupLookAt accordingly.

Glancer

Glancers perform fine orientation adjustments on top of the camera aim to create reactive or cinematic behaviors. Several implementations exist.

ElasticGlancer allows input-driven or touch-driven rotation offsets, letting players drag to rotate the camera slightly. It processes touch or mouse events using the new Unity input system and then computes relative deltas to create a subtle look around effect.

PointOfInterestGlancer blends the camera orientation toward a registered point of interest such as a nearby object or NPC using constraints like maximum glance angles and optional blending curves. It is driven by a PointOfInterestDirector on the target focus.

The Glancer Aim method is called each frame to modify SetupGlance to apply a local rotation to the camera in addition to the aim.

Constraint

Constraints limit the camera goal to satisfy gameplay boundaries. For example,

MinDistanceConstraint ensures the camera does not get too close to the focus object by enforcing a minimum distance.

Other constraints could include colliders to keep the camera outside of walls or volumes to clamp the camera movement to specific zones

Constraints are executed after the planners so they can correct or limit what the planner wrote into SetupGoal

Setup Structure

Each frame a Setup structure is passed through the pipeline It contains

Focus the transform of the main tracked subject

Goal the target camera position

LookAt the point to aim toward

Glance a local rotation offset

These values flow through planners framers glancers and constraints in a predictable order

GoalPlanners set the Goal

Framers set the LookAt

Glancers modify the Glance

Constraints may adjust the Goal after planners

The final camera transform is determined by setting its position to SetupGoal and its rotation by looking at SetupLookAt with the applied SetupGlance rotation

Settings Components

Settings components provide aspect ratio dependent overrides for specific camera elements so designers can quickly adjust offsets or heights for various screen sizes without modifying code For example

LookAheadFramerSettingsComponent maps a data asset to a LookAheadFramer so its offsets change with aspect ratio

HeightOffsetGoalPlannerSettingsComponent maps a data asset to a HeightOffsetGoalPlanner to adjust its vertical offset

They work by applying settings to their respective components during initialization

Supporting Components

SmoothBezierCurve is used to visually place and edit splinebased paths in world space These paths can be used for example by NPCs or camera rails It provides tools to interpolate positions or find the closest point on a curve

CompoundSwitch is a logic utility that allows you to group multiple switches together with AND or OR logic This is sometimes used to drive triggers for cameras enabling them only if a compound set of conditions is true

Typical Camera Flow

When the scene loads a CameraController becomes active activating its planners framers and glancers
When the focus changes or transitions happen the CameraController disables or enables transitions to other CameraControllers If transitions are defined they use the configured CameraControllerTransition to blend to the new camera position and aim

Each frame

The Goal is planned

Constraints correct the Goal

The LookAt is framed

The Glance is applied

Finally the camera transform is updated to match these computed values

Extension Guide

To extend this system for a new gameplay area

Create a new GoalPlanner subclass if you need a new camera follow path or vertical offset

Create a new Framer subclass if you want to change the look at logic for example focusing on a secondary target

Create a new Glancer subclass if you want custom subtle glance behavior like screen shake or mouse aiming

Add a new CameraController for the new area

Link transitions between controllers with CameraControllerTransition

This modular approach ensures you can maintain consistent transitions and reuse common camera behaviors without rewriting the entire logic

Summary

The Club Penguin Island cinematography system uses separate specialized scripts working together to position aim and rotate the camera in a flexible data driven way This makes it easier to expand or adjust cameras for new levels or areas in the game with minimal code changes by simply combining different planners framers glancers and constraints into new CameraController setups